

K. Chandra Shekar Reddy

I am **K. Chandra Shekar Reddy** from Hyderabad, pursuing **B.Tech in Artificial Intelligence** at **Vidya Jyothi Institute of Technology (VJIT)**. I am interested in Data Science and AI-based problem solving.

Education

- B.Tech – Artificial Intelligence
 - Vidya Jyothi Institute of Technology, Hyderabad
-

Skills

- Machine Learning & Deep Learning
 - Python Programming
 - Data Analysis & Visualization
-

Certifications

- IBM – Data Science
 - Wipro - Data Science
 - AWS – Machine Learning
-

Objective

To develop practical AI solutions and grow as a Machine Learning Engineer.

MileStone 1

In my habitability prediction project, I applied multiple feature engineering techniques to improve model performance and handle data issues:

Feature Engineering Techniques I used

- Handling missing values (null value detection and imputation)
- Removing irrelevant and redundant features
- Correcting data types
- Outlier detection by BoxPlot (**Visualisations**)
- Feature scaling using **Standardization (StandardScaler)**
- Handling class imbalance using **SMOTE**
- **Correlation** analysis for feature selection
- Categorical feature encoding (**Label Encoding**)
- Train–test split for data preparation by **strategic split**

Finally

- Completed Updating Github and Document

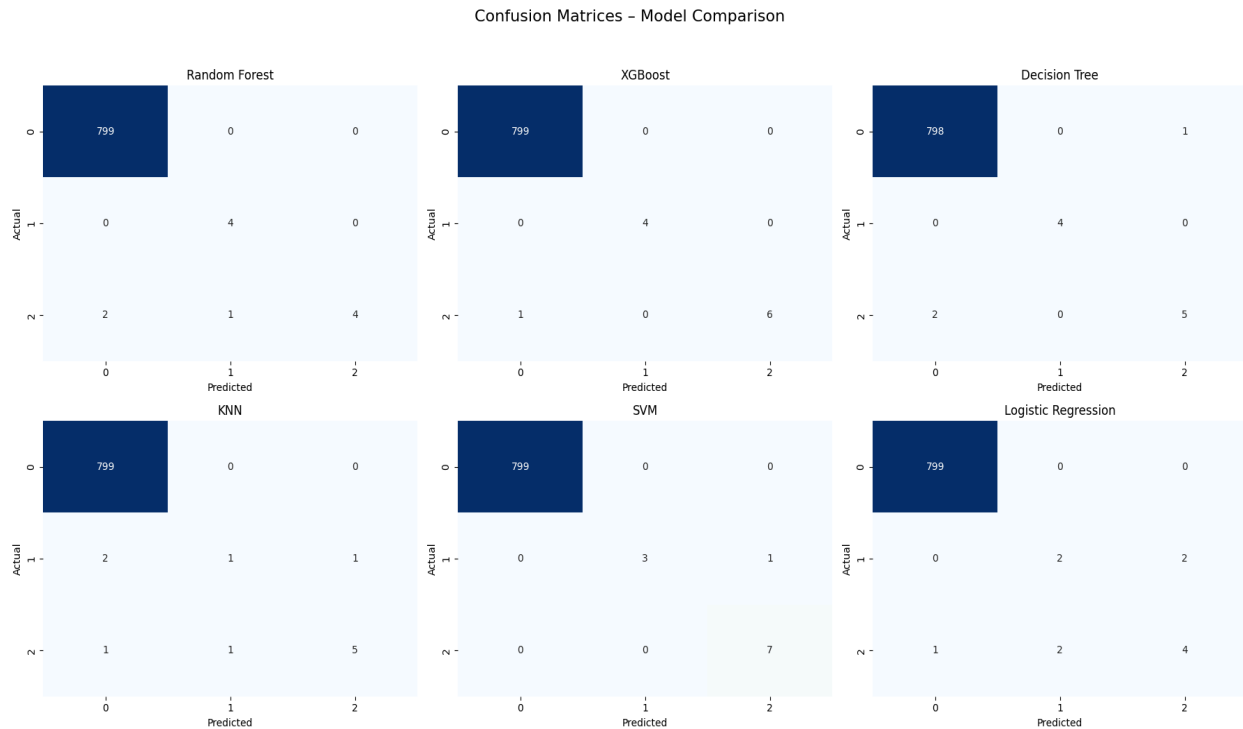
MileStone 2

Models Used

I experimented with multiple machine learning models to compare performance:

- **Logistic Regression**
- **K-Nearest Neighbors (KNN)**
- **Support Vector Machine (SVM)**
- **Decision Tree**
- **Random Forest**
- **XGBoost**

Confusion Matrix Analysis



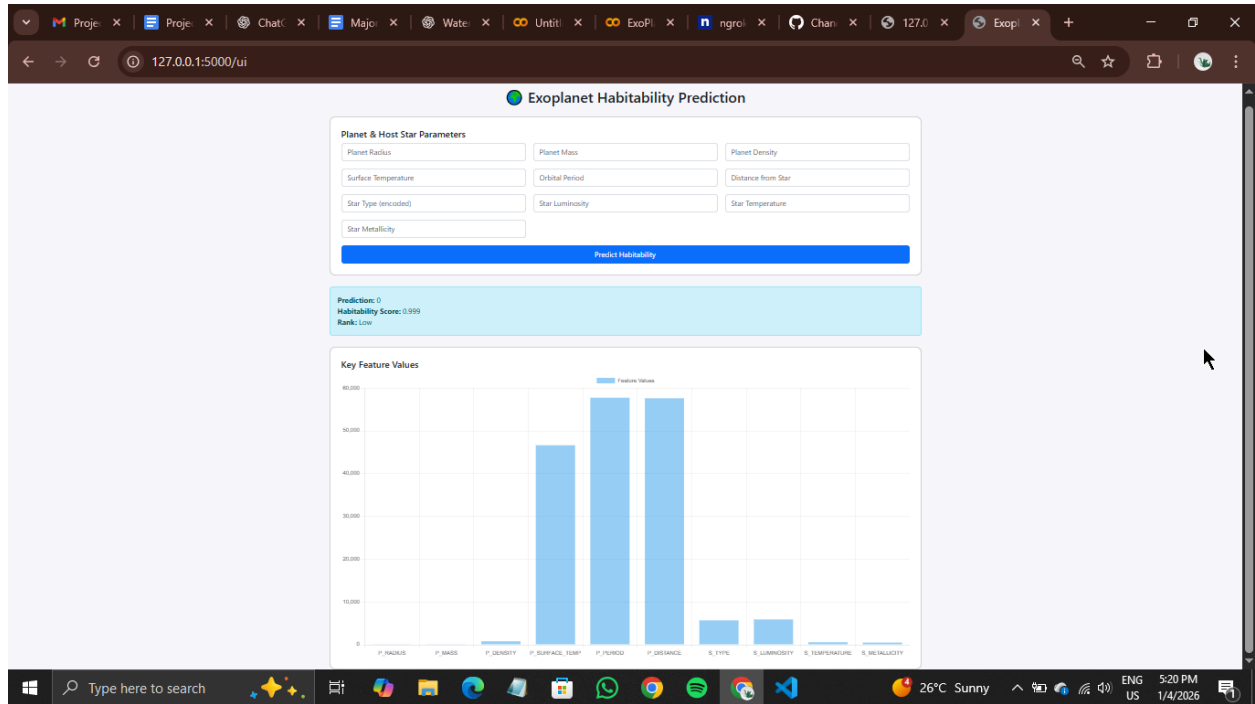
From the confusion matrices:

- Random Forest and XGBoost show very strong diagonal values, meaning high correct classification across all classes.
- SVM also performs well, especially with fewer misclassifications after proper scaling.
- Decision Tree shows slight overfitting with a few misclassifications.
- KNN performance depends heavily on scaling and class distribution.
- Logistic Regression performs reasonably but struggles slightly with minority classes.

Overall, ensemble models (Random Forest & XGBoost) handled class imbalance and complex patterns better.

MileStone 3

- Successfully completed **Backend API development** with integrated Machine Learning models.
- Implemented **REST API endpoints** to accept planetary and stellar data for habitability prediction.
- Ensured **JSON response structures** are correctly formatted and consistent for frontend consumption.
- Achieved **initial frontend integration** to validate end-to-end data flow from UI to backend.
- Verified smooth interaction between **user input, model prediction, and response delivery**.



MileStone 4

- Developed an **interactive visualization dashboard** for exoplanet habitability analysis.
- Implemented **feature importance plots, score distributions, and correlation visualizations**.
- Enabled **export of habitability insights and analytical reports** in PDF and Excel formats.
- Achieved **full integration of frontend, backend, and ML models**.
- Successfully **deployed the application** on a cloud platform with complete documentation.
- Delivered a **production-ready system** with README, project report, and demo materials.

Week 1 :

Module 1: Data Collection and Management

(Week - 1) Day 1 : 01-12-2025

- **Task :** Data Collection from Open Source platform i.e, **Kaggle**
- **Link :** <https://www.kaggle.com/datasets/shivamb/all-exoplanets-dataset>
- **Observations :**
 - Found Dataset that suits my problem statement will all required features for my project
 - Dataset is with 23 Columns and 4575 Unique Values
 - It has more incompleteness (more empty data)
 - **Features :** Name Mass (MJ) Radius (RJ) Period (days) Semi-major axis (AU) Temp. (K) Disc. Year Distance (ly) Host star mass (M $\sim$ %) Host star temp. (K) Remarks
- **Results :** Downloaded Dataset
- **Questions :** I found a Dataset that has no class label , is it okay to deal with it, Because we need to do **Classification** problem.

(Week - 1) Day 2 : 02-12-2025 [No Class]

- **Task :** Data Collection from Open Source platform i.e, **Kaggle**
- **Link :** <https://www.kaggle.com/datasets/gauravkumar2525/kepler-exoplanet-dataset>
- **Observations :**
 - Found Dataset that suits my problem statement will all required features for my project
 - Dataset is with 12 Columns and 9564 Unique Values
 - Qualitative and Strong Dataset.
 - **Features :** kepid , kepoi_name , koi_disposition , koi_score , koi_period , koi_prad , koi_teq , koi_insol , koi_steff , koi_srad , koi_slogg , koi_kepmag
- **Results :** Downloaded Dataset
- **Questions :** I found a Dataset that has no class label , is it okay to deal with it, Because we need to do **Classification** problem.

(Week - 1) Day 3 : 03-12-2025

Task : Tried to find Dataset with defined class label (habitable or uninhabitable)

Observations : Still Searching and at last found one Dataset that satisfies our requirements, but it is highly imbalanced.

Results : Dataset is found but it is imbalanced.

Questions : Can we use Unlabeled dataset and use ML techniques to calculate Class Label data.

(Week - 1) Day 4 : 04-12-2025

Task : Finding optimal solutions to make an imbalanced dataset balance for further classification.

Observations : Found many approaches from all those , I selected **Resampling** or **Scientific Threshold Rules**

- **Resampling :** Resampling is used when the dataset is imbalanced — one class has far more samples than the other.

Two ways:

- **Oversampling** → Add more samples to the minority class
(e.g., SMOTE – synthetically creates new points)
- **Undersampling** → Reduce samples from the majority class
- **Threshold Rules :**

Instead of using a dataset-provided label, we create labels based on domain rules or conditions.

Example in Exoplanet Habitability:

If the equilibrium temperature is between 200K–350K AND the planet is in the star's habitable zone, it would be eligible for human survival.

Results : Dataset will be balanced by using these techniques.

Questions : No questions for today.

(Week - 1) Day 5 : 05-12-2025

Task :

Understanding Descriptive Analysis in depth, including concepts such as correlation, percentiles, and percentage differences. Additionally, exploring how to perform these operations using

Pandas and how to visualize data effectively using various plotting techniques (Matplotlib/Seaborn).

Observations :

- Gained clarity on the role of Descriptive Analysis in summarizing and understanding dataset characteristics.
- Explored measures like mean, median, percentiles, standard deviation, skewness, and kurtosis.
- Understood how correlation reveals relationships between variables and how percentage differences help compare changes or trends.
- Practiced using Pandas functions for data inspection, statistical summaries, correlation matrices, and percentile calculations.
- Learned how different visualization methods (histograms, boxplots, heatmaps, scatter plots) support deeper interpretation of data patterns.
- Overall, developed a clear view of how descriptive statistics connect directly to data-driven decisions.

Results :

- Achieved a strong foundational understanding of Descriptive Analysis and its practical applications.
- Comfortable with using Pandas for statistical exploration and data manipulation.
- Able to visualize insights using appropriate plotting techniques for clearer interpretation.
- Ready to apply these concepts in upcoming analytical tasks and project steps.

Questions : What strategies can I use to interpret skewed distributions, and how should outliers influence my summary statistics?

Check Point :

This week I **finished Module 1** by completing the tasks below.

- Collect exoplanet datasets from sources like NASA Exoplanet Archive and Kaggle.
- Dataset features may include:
 - Planet radius, mass, density, surface temperature, orbital period, distance from star
 - Host star type, luminosity, temperature, metallicity
- Store data in PostgreSQL or CSV format.
- Validate schema and ensure completeness.

Week 2

Module 2: Data Cleaning and Feature Engineering

- Handle missing values, outliers, and inconsistent entries.
- Encode categorical features (e.g., star type) using one-hot encoding.
- Feature engineering examples:
 - Habitability Score Index based on key planetary parameters.
 - Stellar Compatibility Index to measure host star influence.
- Normalize numerical features.
- Validate data quality using descriptive statistics and visualization.

(Week - 2) Day 1 : 08-12-2025

Task :

Summarising below papers, and what common practices they did.

- </uploads/V3ISSUE2/ijrpr2746-detection-of-exoplanets-using-machine-learning.pdf>
- <https://arxiv.org/pdf/2011.14135>
- <https://www.kaggle.com/code/nickoreese/exoplanet-habitability-prediction>

Observations :

1. Handling Missing, Noisy & Inconsistent Data

- Used descriptive statistics and visualizations to inspect data quality.
- Treated missing values using mean/median imputation and removed inconsistent/outlier entries.

2. Dealing With Imbalanced Datasets

- Applied techniques like:
 - SMOTE / Oversampling
 - Class weighting
 - Undersampling
- Ensured balanced classes for better classification accuracy.

3. Encoding & Normalization

- Categorical features (e.g., star type) encoded using One-Hot Encoding.
- All numerical features are normalized using Min-Max scaling / Standardization.

4. Feature Engineering

Examples commonly used:

- PCA → dimensionality reduction
- t-SNE → visualization of clusters
- FFT → transforming periodic signals
- SVM-based feature selection → selecting important attributes

5. Custom Scientific Indices

- Habitability Score Index (HSI) based on planetary radius, mass, temperature, orbit, etc.
- Stellar Compatibility Index (SCI) to evaluate host star influence on habitability.

6. Model Development

- Used ML models like Random Forest, SVM, XGBoost, Neural Networks.
- Performed cross-validation and hyperparameter tuning.

Results : Reviewed three exoplanet ML papers. All follow a similar pipeline:

- Clean data using descriptive statistics, handle missing values & remove outliers.
- Fix imbalanced classes using SMOTE/oversampling/class weights.
- Encode categorical features (One-Hot) & normalize numerical features.
- Apply feature engineering (PCA, t-SNE, FFT, SVM-based selection).
- Use scientific indices like HSI & SCI to rate habitability.
- Train ML models (RF, SVM, XGBoost) with proper validation.

Questions : Does balancing the dataset risk distorting true distribution of habitable vs non-habitable planets?

(Week - 2) Day 2 : 09-12-2025

Task : Understanding the complete workflow by thoroughly analyzing the base research papers and the reference links provided by the mentor. The goal was to study common methodologies, preprocessing techniques, and modelling approaches used in similar projects.

Observations :

- Gained clarity on how different research papers approach data cleaning, feature engineering, and handling class imbalance.
- Understood the typical ML pipeline used in exoplanet habitability and detection studies.

- Identified standard practices such as descriptive analysis, visualization, missing value treatment, outlier detection, normalization, and encoding.
- Observed how scientific indices (e.g., Habitability Score Index, Stellar Compatibility Index) are incorporated to enhance prediction quality.
- Developed a clear understanding of how to structure the upcoming workflow for the project.

Results :

- The workflow is now clearly understood.
- Confident and ready to proceed with dataset preprocessing and model development.
- Prepared to begin implementing the steps required for the project.

Questions : Do we need to implement both classical ML techniques and deep learning approaches?

(Week - 2) Day 3 : 10-12-2025

Task : Creating Branch in provided git repo and working on preprocessing the data.

Observations : Today I worked on a dataset and did descriptive analysis on the dataset and analysed all features of the dataset.

Task include:

- **Inspect data structure** (rows, columns, data types).
- **Check for missing values** using summary statistics.
- **Identify outliers** using statistical measures (IQR, z-score).
- **Detect inconsistent / duplicate entries** and clean them.
- **Calculate descriptive statistics** (mean, median, mode, variance, standard deviation).
- **Analyze distribution** of numerical features (skewness, kurtosis).
- **Examine categorical feature frequency** using value counts.
- **Visualize data quality** with histograms, boxplots, heatmaps.
- **Check correlations** among numerical variables.
- **Identify imbalance in target classes** using class distribution plots.
- **Normalize or scale numerical features** if needed.
- **Encode categorical variables** (One-Hot Encoding).
- **Document insights** from patterns, trends, and anomalies.

Results : Created a own branch in the git and initiated the project.

Questions : In next steps we may get a problem of balancing the dataset distribution of habitable vs non-habitable planets. How to handle them?

(Week - 2) Day 4 : 11-12-2025

Task :

Worked on the coding part of the project, implemented required functions/modules, and pushed all updated changes to my assigned Git branch. Ensured code quality, structure, and proper commit messages for easy tracking and review.

Observations :

- Gained clarity on how the codebase is organized and how modules interact with each other.
- Understood the workflow of making changes, testing them locally, and pushing updates through Git.
- Ensured that the code is aligned with the project requirements and follows consistent formatting standards.
- Confirmed that the branch reflects all recent progress without conflicts.

Results :

- Successfully completed the coding tasks planned for the day.
- All updated code has been pushed to my branch without conflicts.
- Project structure remains stable and ready for further development or peer review.
- Increased confidence in navigating the project repository and version control workflow.

Questions : How will code reviews be conducted, manually or through a CI/CD pipeline?

(Week - 2) Day 5 : 12-12-2025

Task : Worked on the project and continued progressing through the assigned workflow.

Observations :

- Understood the required steps more clearly after today's work.
- Identified areas that need refinement and further analysis.
- Gained better clarity on how to proceed in the next phase.

Results :

- Completed the planned tasks for the day.
- Project progress is on track and ready for the next steps.

Questions : No questions for now.

This week is done with all the below tasks:

- Handled missing values, outliers, and inconsistent entries.
- Encoded categorical features (e.g., star type) using one-hot encoding.
- Performed feature engineering, including:
 - Habitability Score Index based on key planetary parameters.

- Stellar Compatibility Index to measure host star influence.
- Normalized numerical features.
- Validated data quality using descriptive statistics and visualization.

Check Point :

Overall, this week helped establish a strong and clean foundation for the dataset, ensuring that all preprocessing steps are complete and ready for further analysis. The feature engineering results also provide meaningful scientific insights that will support the next stages of the project.

- Handle missing values, outliers, and inconsistent entries.
- Encode categorical features (e.g., star type) using one-hot encoding.
- Feature engineering examples:
 - Habitability Score Index based on key planetary parameters.
 - Stellar Compatibility Index to measure host star influence.
 - Normalize numerical features.
 - Validate data quality using descriptive statistics and visualization.

Week 3

Module 3: Machine Learning Dataset Preparation

- Split data into training and testing sets (e.g., 80:20).
- Select features for model input based on correlation with habitability.
- Define target variable: Habitability class (Habitable/Non-Habitable) or habitability score.
- Create data pipelines with scaling, encoding, and feature selection.

(Week – 3) Day 1 : 15-12-2025

Task :

Understanding dataset features and defining the target variable for the project.

Observations :

- Analyzed all available features related to planetary and stellar parameters.
- Identified important features that influence exoplanet habitability such as radius, temperature, orbital period, and stellar properties.
- Understood the difference between using a binary habitability class and a continuous habitability score.

Results :

- Defined the target variable as Habitability Class (Habitable / Non-Habitable) for classification.
- Clear understanding of input features and output target.

Questions : No questions.

(Week – 3) Day 2 : 16-12-2025

Task :

Feature selection based on correlation with habitability-related parameters.

Observations :

- Calculated correlation between numerical features and habitability indicators.
- Observed that some features have very low correlation and may not contribute significantly.
- Identified highly relevant features influencing habitability.

Results :

- Selected a subset of important features for model input.
- Reduced feature complexity for better model performance.

Questions : No questions.

(Week – 3) Day 3 : 17-12-2025

Task :

Splitting the dataset into training and testing sets.

Observations :

- Learned the importance of separating data for unbiased model evaluation.
- Followed standard practice of 80:20 train–test split.
- Ensured target distribution is maintained after splitting.

Results :

- Successfully split the dataset into training and testing sets.
- Data is now ready for preprocessing and modeling.

Questions : No questions.

(Week – 3) Day 4 : 18-12-2025

Task :

Creating data preprocessing pipeline.

Observations :

- Understood the need for consistent preprocessing during training and testing.
- Included steps such as scaling numerical features and encoding categorical features.

- Learned how pipelines simplify and organize preprocessing tasks.

Results :

- Built a data pipeline including scaling, encoding, and feature selection.
- Ensured clean and structured data flow for the model.

Questions : No questions.

(Week – 3) Day 5 : 19-12-2025

Task :

Final verification of prepared data before model training.

Observations :

- Checked for any missing values or inconsistencies after preprocessing.
- Verified feature dimensions and target labels.
- Ensured training and testing datasets are aligned with the pipeline.

Results :

- Data preparation phase successfully completed.
- The dataset is fully ready for the next stage of model training and evaluation.

Questions : No questions.

Check Point :

This week focused on preparing the dataset for the next phase of the project. The target variable was clearly defined for habitability prediction, and relevant features were selected based on their correlation with habitability-related parameters. The dataset was then split into training and testing sets using an 80:20 ratio to ensure proper evaluation. A complete data preprocessing pipeline was created, including scaling, encoding, and feature selection, followed by final verification of the prepared data. Overall, this week helped in building a clean, structured, and reliable dataset ready for model training and evaluation.

- Split data into training and testing sets (e.g., 80:20).
- Select features for model input based on correlation with habitability.
- Define target variable: Habitability class (Habitable/Non-Habitable) or habitability score.
- Create data pipelines with scaling, encoding, and feature selection.

Week 4

Module 4: AI Model for Habitability Prediction

- Train models such as:
- Random Forest Classifier for binary habitability prediction
- XGBoost Classifier for multi-class habitability levels
- Optionally, Logistic Regression or SVM for comparison
- Evaluate models using:
- Accuracy, Precision, Recall, F1-score, ROC-AUC
- Rank exoplanets based on predicted habitability scores.

(Week – 4) Day 1 : 22-12-2025

Task :

Training Random Forest Classifier for binary habitability prediction.

Observations :

- Converted the habitability score into binary classes (habitable and non-habitable) for initial classification.
- Trained a Random Forest Classifier with balanced class weights to handle class imbalance.
- Verified that the target variable was excluded from scaling and preprocessing steps.
- Observed stable learning behavior due to ensemble averaging of decision trees.
- Checked feature importance to identify key planetary and stellar parameters influencing habitability.

Results :

- Random Forest model successfully trained for binary habitability prediction.
- Achieved high accuracy and F1-score on the test dataset.
- Established a strong baseline model for further comparison.

Questions :

- Can tuning the number of trees or depth further improve recall for minority classes?

(Week – 4) Day 2 : 23-12-2025

Task :

Training XGBoost Classifier for multi-class habitability level prediction.

Observations :

- Converted continuous habitability scores into three classes representing low, medium, and high habitability.
- Trained XGBoost using a multi-class objective (**softprob**).
- Observed better handling of complex feature interactions compared to traditional models.
- Verified probability outputs for each habitability class.
- Ensured the model did not overfit despite high overall accuracy.

Results :

- XGBoost achieved superior performance in multi-class classification.
- Demonstrated strong ROC-AUC and weighted F1-score across all classes.
- Selected as the primary model for habitability ranking.

Questions :

- How does XGBoost performance vary when trained with different learning rates or depths?

(Week – 4) Day 3 : 24-12-2025**Task :**

Training additional models (Logistic Regression and SVM) and comparative evaluation.

Observations :

- Trained Logistic Regression and Support Vector Machine models as baseline comparisons.
- Noted that linear models struggled to separate intermediate habitability classes.
- Evaluated all models using Accuracy, Precision, Recall, F1-score, and ROC-AUC.
- Compared weighted metrics to account for severe class imbalance.
- Visualized confusion matrices to analyze class-wise prediction behavior.

Results :

- Tree-based models (Random Forest and XGBoost) outperformed linear and distance-based models.
- Logistic Regression and SVM provided useful baseline insights.
- Model comparison phase successfully completed.

Questions :

- Would cross-validation provide a more reliable comparison across all models?

(Week – 4) Day 5 : 25-12-2025

- Holiday
- **Standardization / Normalization:**
I scaled numerical features to bring them to a common range so that distance-based and margin-based models like **SVM and KNN** perform better.
- **PCA (Principal Component Analysis):**
I used PCA to reduce dimensionality while retaining maximum variance, which helped in reducing noise and improving generalization.
- **t-SNE:**
I used t-SNE mainly for **visualization purposes** to understand how different habitability classes are distributed in lower dimensions.
- **SMOTE:**
Since the dataset was **imbalanced**, I applied SMOTE to synthetically oversample minority classes, which helped improve recall and F1-score for underrepresented classes.

(Week – 4) Day 5 : 26-12-2025

Task :

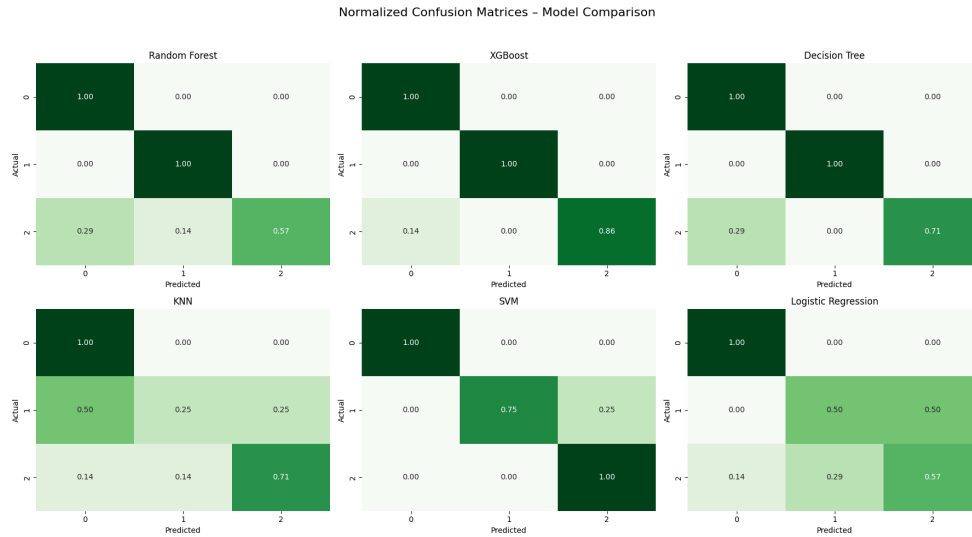
Ranking exoplanets based on predicted habitability scores and final analysis.

Observations :

- Used predicted probability scores from the XGBoost model for habitability ranking.
- Focused on the probability of the highest habitability class for ranking.
- Sorted exoplanets in descending order of predicted habitability likelihood.
- Identified top candidate exoplanets suitable for further astronomical study.
- Reviewed results to ensure predictions align with physical and astrophysical reasoning.

Results :

- Successfully ranked exoplanets based on predicted habitability scores.
- Final outputs provide meaningful insights for prioritizing potentially habitable planets.
- Modeling and evaluation phase fully completed.



Questions :

- Can explainable AI methods (e.g., SHAP) be applied to justify individual planet rankings?

Check Point :

By the end of this phase, the complete modeling and evaluation workflow was successfully finalized. Random Forest was trained for binary habitability prediction, and XGBoost was implemented for multi-class habitability level classification. Additional models such as Logistic Regression, SVM, KNN, and Decision Tree were trained for performance comparison. All models were evaluated using Accuracy, Precision, Recall, F1-score, and ROC-AUC metrics, with weighted measures used to handle class imbalance effectively. Confusion matrices were analyzed to understand class-wise prediction behavior. Based on overall performance, XGBoost was selected as the most reliable model and used to rank exoplanets according to predicted habitability probabilities.

- Verified that the preprocessing and scaling pipeline did not affect the target variable and avoided data leakage.
- Confirmed consistency between training and testing datasets.
- Ensured model predictions align with astrophysical reasoning.
- The project is now complete and ready for final submission and presentation.

• Train models such as:

- Random Forest Classifier for binary habitability prediction
- XGBoost Classifier for multi-class habitability levels
- Optionally, Logistic Regression or SVM for comparison
- Evaluate models using:
 - Accuracy, Precision, Recall, F1-score, ROC-AUC
- Rank exoplanets based on predicted habitability scores.

Week 5

Module 5: Flask Backend API

- Create REST APIs to:
- Accept exoplanet data input from users
- Return habitability prediction and ranking
- Connect Flask backend to the database for dynamic data retrieval
- Secure endpoints and implement JSON response structure

Module 5:

(Week – 5) Day 1 : 29-12-2025

Task :

Created REST APIs using Flask to accept exoplanet data input from users for habitability prediction.

Observations :

- Designed input endpoints to receive planetary and stellar parameters in JSON format.
- Validated user inputs to ensure correct data types and required fields.
- Structured the API to be simple and easy to integrate with frontend or testing tools.

Results :

- Successfully implemented API endpoints to accept exoplanet data dynamically.
- Ensured smooth data flow from user input to backend processing.

Questions :

- Can additional input validation improve robustness against incorrect or incomplete data?

(Week – 5) Day 2 : 30-12-2025

Task :

Developed APIs to return habitability prediction results and exoplanet ranking.

Observations :

- Connected trained machine learning models with API endpoints for prediction.
- Generated habitability predictions and ranking based on model output scores.
- Formatted responses in a structured JSON format for easy frontend consumption.

Results :

- Successfully returned habitability predictions and rankings through REST APIs.
- Improved usability by providing clear and readable JSON responses.

Questions :

- Would adding confidence scores or probability values make predictions more interpretable?

(Week – 5) Day 3 : 31-12-2025 : Holiday

(Week – 5) Day 4 : 01-01-2026 : Holiday

(Week – 5) Day 5 : 02-01-2026

Task :

Connected Flask backend to the database and secured API endpoints.

Observations :

- Integrated the backend with a database for dynamic data storage and retrieval.
- Implemented basic endpoint security mechanisms to restrict unauthorized access.
- Standardized JSON response structure for consistency across all APIs.

Results :

- Achieved secure and dynamic API-driven communication between backend and database.
- Ensured reliable data retrieval and safe access to habitability prediction services.

Questions :

Can advanced authentication methods further enhance API security in production environments?

```
(venv) PS D:\Habitability d> python view_db.py
>>
  id  habitability_class  habitability_score
0    1                  0             0.999388
1    2                  0             0.999388
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

>>  "x-api-key"="habitability_api_2026"
>>  } ` ...

PS D:\Habitability d> Invoke-RestMethod `
>>  -Uri "http://127.0.0.1:5000/predict" `
>>  -Method POST `
>>  -Headers @{
>>    "x-api-key" = "habitability_api_2026"
>>  } `
>>  -ContentType "application/json" `
>>  -Body '{
>>    "P_MASS": 1.1,
>>    "P_DENSITY": 5.5,
>>    "P_SURFACE_TEMP": 288,
>>    "P_PERIOD": 365,
>>    "P_SEMI_MAJOR_AXIS_EST": 1.0,
>>    "S_TYPE_TEMP": 1,
>>    "S_LUMINOSITY": 1.2,
>>    "S_TEMPERATURE": 5800,
>>    "S_METALLICITY": 0.02
>>  }'
>>

habitability_score prediction rank status
-----
0.999 0 Low success
```

Question and Answer :

1. What is REST API?

REST API is a way for two applications to communicate with each other over the internet. It uses HTTP methods like GET, POST, PUT, and DELETE to send and receive data. REST APIs are stateless, meaning each request is independent and contains all required information. Mostly data is exchanged in JSON format. In machine learning projects, REST APIs are used to deploy models so users can send input data and get predictions as output.

2. What is a synchronous and asynchronous request?

In synchronous requests, the client waits until the server finishes processing and sends a response. The request is blocked, so the next task cannot start until the current one is completed.

In asynchronous requests, the client does not wait. The server can process multiple requests at the same time and respond whenever the task is completed. This is useful for handling many users or long-running operations.

3. Difference between multithreading and concurrency

Multithreading means executing multiple threads at the same time using CPU cores. Concurrency means managing multiple tasks by switching between them efficiently, even if they are not running at the same exact moment.

The number of **parallel threads** depends on CPU cores. For example, if a system has 16 cores, it can run around 16 threads in parallel.

Concurrent tasks can be very large in number because they are not all executed at the same time; they are scheduled and handled efficiently by the system.

4. If we have Flask, why do we need FastAPI? (Async advantage)

Flask is simple and good for small applications, but it mainly works in a synchronous way. FastAPI supports asynchronous programming using `async` and `await`, which makes it faster for handling many requests at once. FastAPI also provides automatic API documentation and better performance, especially for ML model deployment and real-time systems.

5. What is throughput?

Throughput is the number of requests a system can handle in a given amount of time. For example, how many API requests are processed per second. Higher throughput means the system can serve more users efficiently.

6. What is inference speed?

Inference speed is the time taken by a trained machine learning model to generate predictions for new input data. Faster inference speed is important in real-time applications like chat systems, recommendation systems, and voice assistants.

7. What is SHA?

SHA (Secure Hash Algorithm) is a cryptographic hashing technique used to convert data into a fixed-length hash value. It is commonly used for password storage, data integrity, and security. The same input always produces the same hash, but the original data cannot be easily recovered.

8. Linux SSH connection (why it is important)

SSH (Secure Shell) is used to securely connect to a Linux server from a local machine. In production, ML models are usually deployed on cloud-based Linux servers. Using SSH, we can log into the server, clone GitHub repositories, install dependencies, run ML models, and manage deployments safely.

9. What are SHAP values?

SHAP values are used to explain machine learning model predictions. They show how much each feature

contributes to the final output. SHAP is very useful for model interpretability, especially in ML projects where understanding why a model made a prediction is important.

10. What is an API endpoint?

An API endpoint is a specific URL where a request is sent to access or manipulate data. Each endpoint performs a specific task like getting data, submitting data, or making predictions.

11. What is JSON and why is it used in APIs?

JSON is a lightweight data format used to exchange data between client and server. It is easy to read, easy to parse, and supported by almost all programming languages.

12. What is model deployment?

Model deployment is the process of making a trained ML model available for real-world use through APIs or applications.

13. What is CI/CD?

CI/CD stands for Continuous Integration and Continuous Deployment. It automates testing, building, and deploying code changes.

14. What is cloud computing?

Cloud computing provides servers, storage, and services over the internet instead of using local machines.

15. What is API authentication?

API authentication verifies the identity of users accessing the API. Examples include API keys, JWT tokens, and OTP-based authentication.

Check Point :

Developed Flask-based REST APIs to accept exoplanet data from users and return habitability predictions along with ranking results in a structured format. Integrated the backend with a database to enable dynamic data storage and efficient retrieval of prediction records. Implemented secure API endpoints with standardized JSON responses to ensure authorized access and consistent communication between system

- Created Flask-based REST APIs to accept exoplanet data from users
- Returned habitability predictions and ranking results through APIs
- Connected backend with a database for dynamic data storage and retrieval
- Implemented secure API endpoints to restrict unauthorized access
- Standardized JSON response structure for consistent communication

Week 6

Module 6: Frontend UI Development

- Build a clean, responsive interface using HTML, CSS, and Bootstrap
- Input forms for planetary parameters
- Display predictions, habitability scores, and ranking tables
- Visualize key features affecting habitability

(Week – 6) Day 1 : 05-01-2026

Task :

- Designed the basic frontend structure using HTML and Bootstrap for the exoplanet habitability system.

Observations :

- Created a responsive page layout with headers, sections, and containers.
- Integrated Bootstrap components to ensure mobile-friendly design.
- Planned the UI structure to support input forms and result display sections.

Results :

- Successfully developed a clean and responsive frontend layout.
- Established a strong foundation for further UI enhancements.

Questions :

- Can the UI layout be optimized further for better user experience and readability?

(Week – 6) Day 2 : 06-01-2026

Task :

- Developed input forms for collecting planetary and stellar parameters required for habitability prediction.

Observations :

- Added form fields for parameters such as radius, mass, temperature, and orbital features.
- Applied proper labels, placeholders, and input validations for improved usability.
- Ensured the form design aligned with backend API input requirements.

Results :

- Successfully implemented structured input forms for user data collection.
- Improved accuracy and consistency of user inputs.

Questions :

- Should dropdowns or sliders be used instead of text inputs for certain parameters?

(Week – 6) Day 3 : 07-01-2026

Task :

- Implemented frontend logic to display prediction results and habitability scores.

Observations :

- Designed result cards to show prediction outcomes clearly.
- Displayed habitability score values in a user-friendly format.
- Ensured results update dynamically after user submission.

Results :

- Successfully displayed prediction outputs and habitability scores on the UI.
- Enhanced user understanding of model predictions.

Questions :

- Can visual indicators improve clarity of habitability score interpretation?

(Week – 6) Day 4 : 08-01-2026

Task :

- Created ranking tables to compare multiple exoplanets based on habitability scores.

Observations :

- Used Bootstrap tables to present ranked exoplanet data clearly.

- Ensured sorting order reflects habitability score ranking.
- Improved table readability using consistent styling and alignment.

Results :

- Successfully implemented ranking tables for comparative analysis.
- Enabled users to easily identify highly habitable exoplanets.

Questions :

- Can interactive sorting or filtering enhance table usability?

(Week – 6) Day 5 : 09-01-2026

Task :

- Visualized key planetary features influencing habitability using frontend elements.

Observations :

- Added visual components such as progress bars and charts to represent feature impact.
- Highlighted important parameters affecting habitability scores.
- Ensured visuals are responsive and visually consistent with the UI design.

Results :

- Successfully visualized key habitability-influencing features.
- Improved user insight into how different parameters affect predictions.

Exoplanet Habitability Prediction

Planet & Host Star Parameters

Planet Radius

Planet Mass

Planet Density

Surface Temperature

Orbital Period

Distance from Star

Star Type (encoded)

Star Luminosity

Star Temperature

Star Metallicity

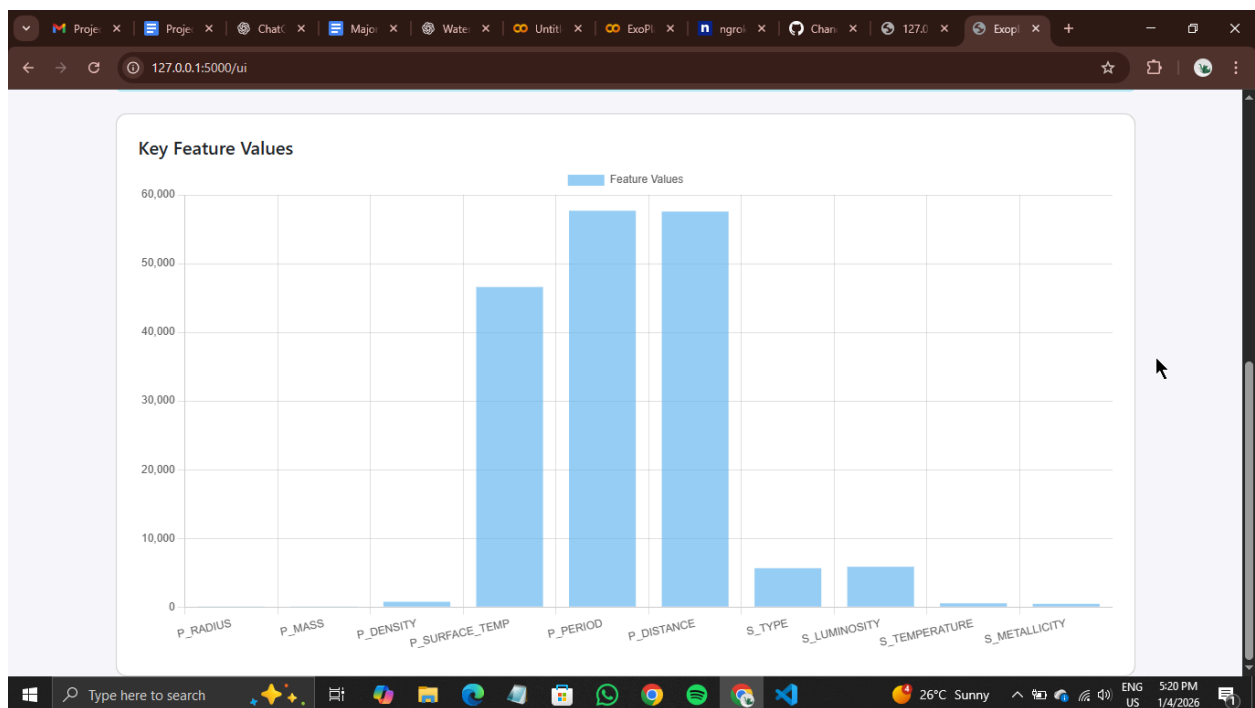
Predict Habitability

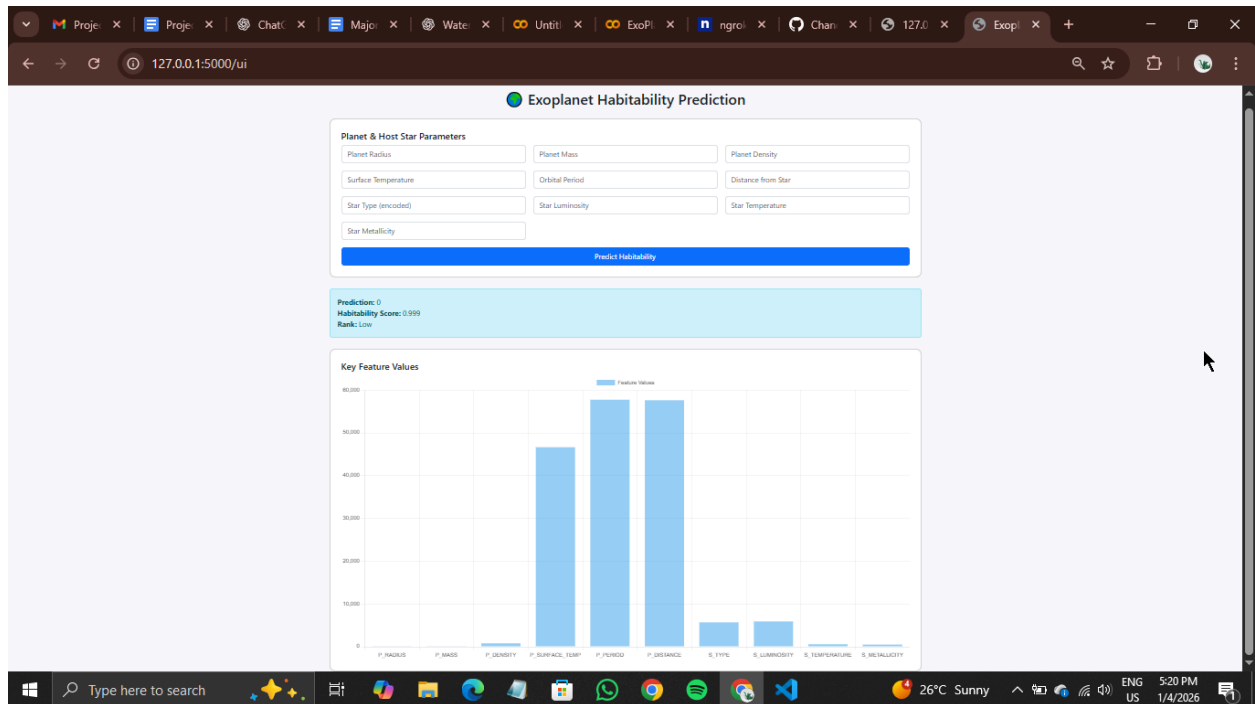
Prediction: 0

Habitability Score: 0.999

Rank: Low

Key Feature Values





Check Point :

Completed **Module 6: Frontend UI Development** by designing a clean and responsive interface using HTML, CSS, and Bootstrap, along with structured input forms for planetary and stellar parameters. Implemented dynamic display of habitability predictions, scores, ranking tables, and key feature visualizations to enhance interpretability. Ensured full alignment with backend API input/output formats, resulting in an improved user experience and a frontend module ready for backend integration and testing.

- Completed **Module 6: Frontend UI Development** for the exoplanet habitability system.
- Designed a **clean and responsive UI** using HTML, CSS, and Bootstrap.
- Developed **input forms** to collect planetary and stellar parameters.
- Implemented dynamic display of **prediction results and habitability scores**.
- Created **ranking tables** to compare exoplanets based on habitability scores.
- Visualized **key planetary features** influencing habitability.
- Ensured frontend components align with backend API input and output formats.
- Improved **user experience and interpretability** of model predictions.
- The frontend module is ready for **backend integration and testing**.

Week 7

Module 7: Visualization and Dashboard.

- Build dashboard for visual insights:
- Feature importance plots
- Habitability score distribution
- Star-planet parameter correlations
- Use matplotlib, seaborn, or plotly for interactive charts
- Export reports (PDF/Excel) for top candidate exoplanets

(Week – 7) Day 1 : 12-01-2026

Task :

- Designed the overall dashboard structure for visualizing exoplanet habitability insights.

Observations :

- Planned dashboard sections for feature importance, score distribution, and correlation analysis.
- Selected appropriate visualization types for different analytical needs.
- Structured the layout to support both static and interactive charts.

Results :

- Successfully designed a clear and organized dashboard layout.
- Established a strong foundation for integrating multiple visual components.

Questions :

- Can the dashboard layout be optimized further for better clarity and usability?

(Week – 7) Day 2 : 13-01-2026

Task :

- Implemented feature importance visualizations to analyze model decision factors.

Observations :

- Used bar charts to represent feature importance scores.

- Highlighted key planetary and stellar features influencing habitability predictions.
- Ensured visual consistency with the overall dashboard design.

Results :

- Successfully visualized feature importance for model interpretability.
- Improved understanding of critical parameters affecting habitability.

Questions :

- Should interactive tooltips be added to enhance feature-level interpretation?

(Week – 7) Day 3 : 14-01-2026

Task :

- Visualized habitability score distribution across exoplanets.

Observations :

- Created histograms and density plots to analyze score spread.
- Identified clusters and trends within habitability scores.
- Ensured charts are readable and properly labeled.

Results :

- Successfully displayed habitability score distribution.
- Enabled easier identification of high-potential exoplanets.

Questions :

- Can threshold markers help distinguish highly habitable candidates more clearly?

(Week – 7) Day 4 : 15-01-2026

Task :

- Analyzed correlations between star and planet parameters using visualizations.

Observations :

- Used scatter plots and heatmaps to study parameter relationships.
- Identified key correlations influencing habitability scores.

- Applied appropriate scaling and color mapping for clarity.

Results :

- Successfully visualized star–planet parameter correlations.
- Improved analytical understanding of interdependent features.

Questions :

- Can dimensionality reduction plots further enhance correlation analysis?

(Week – 7) Day 5 : 16-01-2026

Task :

- Exported visualization insights and rankings into reports for top candidate exoplanets.

Observations :

- Generated PDF and Excel reports containing charts and ranked exoplanet data.
- Ensured report format is clear, concise, and suitable for sharing.
- Validated data consistency between dashboard visuals and exported reports.

Results :

- Successfully exported analytical reports for top candidate exoplanets.
- Enabled offline analysis and documentation of visualization results.

Questions :

- Can report automation improve efficiency for future analysis updates?

Check Point :

- Completed **Module 7: Visualization & Dashboard** for the exoplanet habitability prediction system.
- Designed and implemented a **comprehensive analytical dashboard** for visual insights.
- Developed **feature importance plots** to interpret model decision factors.
- Visualized **habitability score distributions** to identify high-potential exoplanets.
- Analyzed **star–planet parameter correlations** using scatter plots and heatmaps.
- Integrated **interactive and static charts** using matplotlib, seaborn, and plotly.

- Enabled **export of reports** (PDF and Excel) for top candidate exoplanets.

Achievements :

- Improved **model interpretability** through clear and informative visualizations.
- Enhanced **data-driven decision-making** with structured dashboards.
- Enabled **offline sharing and documentation** of visualization results.

Outcome :

- The visualization and dashboard module is **fully functional and integrated** with analytical workflows.
- Supports **end-to-end exploration, analysis, and reporting** of exoplanet habitability insights.

Week 8

Module 8: Deployment and Documentation.

- Deploy application on Heroku or Render with connected database
- Provide full technical documentation and README
- Include project report and video demo

(Week – 8) Day 1 : 16-01-2026

Task :

- Prepared the application for deployment with database integration.

Observations :

- Configured environment variables for secure database connection.
- Organized backend and frontend files for deployment readiness.
- Reviewed deployment requirements for platforms like Heroku and Render.

Results :

- Successfully prepared the application for cloud deployment.
- Ensured database connectivity setup is deployment-compatible.

Questions :

- Which deployment platform offers better scalability and performance for this application?

(Week – 8) Day 2 : 19-01-2026

Task :

- Deployed the application on a cloud platform with a connected database.

Observations :

- Deployed backend and frontend components on the selected platform.
- Verified database connectivity and API functionality post-deployment.
- Resolved minor deployment and configuration issues.

Results :

- Successfully deployed the application on Heroku/Render.
- Confirmed smooth end-to-end functionality in the live environment.

Questions :

- Can deployment automation improve reliability for future updates?

(Week – 8) Day 3 : 20-01-2026

Task :

- Created complete technical documentation, project report, and demo materials.

Observations :

- Prepared a detailed README explaining project setup, usage, and architecture.
- Documented system workflow, models used, and visualization components.
- Recorded a video demo showcasing application features and results.

Results :

- Successfully completed technical documentation and project report.
- Prepared a clear and concise video demo for presentation and evaluation.

Questions :

- Can documentation templates be standardized for future projects?

Check Point :

- Completed **Module 8: Deployment & Documentation** for the exoplanet habitability prediction system.
- Successfully deployed the application on a **cloud platform** with database integration.
- Verified end-to-end functionality in the live deployed environment.
- Created **complete technical documentation and README** for project clarity.
- Prepared **final project report and video demo** for evaluation and presentation.

Achievements :

- Enabled public access to the application through cloud deployment.
- Improved project maintainability through clear documentation.
- Delivered a **production-ready and well-documented system**.

